

SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
CHENNAI – 602105
Department of Computer Science and Engineering

ASSESSMENT TOOL 3 –Dataset Integration Task

Course Code:	DSA0502	Course Title:	Query Processing for Data Science
CO Assessed:	CO3	Assessment:	Tool 3 - Dataset Intégration Task
Weightage:		Total Marks:	
CO3 Statement: Identify and clean inconsistencies in data by handling missing values, duplicates, outliers, formatting issues, and applying techniques like fuzzy matching and regex. (BL5 and BL6).			
Student Name:	B.Varun Kumar Reddy	Reg. No.:	192324066
Section / Batch:	AIDS 4 th Year	Date of Submission:	13/06/26
Faculty In-charge:	Dr. B.Shyamala Gowri	Project Mode:	Individual Submission

Code of Conduct

I B. Varun Kumar Reddy(192324066) (Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: _____

Dataset Integration Task

Question 3 – Product Dataset Integration

An e-commerce company maintains product information in **Supplier, Warehouse, and Online Store datasets**. Product names, product codes, prices, categories, and stock quantities are represented differently across the datasets. Some records are duplicated or contain spelling errors and invalid product codes.

Task: Develop a Python-based data cleanup and integration solution. Use **RegEx to validate product codes, fuzzy matching to identify similar product names, duplicate detection, standardization of categories, outlier detection for prices, and normalization of numerical values**. Integrate the datasets and generate a final product master file in CSV and JSON formats.

Answer:

Python Code

```
import re
import json
from difflib import SequenceMatcher
import pandas as pd
import numpy as np

# -----
# 1. Sample source datasets (simulating Supplier / Warehouse / Online Store)
# -----
supplier_df = pd.DataFrame({
    "prod_name": ["Wireless Mouse", "Bluetooth Speaker", "USB-C Cable", "Laptop Stand",
"Wireless Mouse "],
    "prod_code": ["PRD-1001", "PRD1002", "prd-1003", "PRD-1004", "PRD-1001"],
    "category": ["Electronics", "electronics", "Accessories", "Accessories", "Electronics"],
    "price": [799, 1499, 199, 899, 799],
    "stock": [120, 45, 300, 60, 120],
})

warehouse_df = pd.DataFrame({
    "product_name": ["Wireless Mouse", "Bluetooth Speker", "USB C Cable", "Laptop Stand",
"Gaming Keyboard"],
    "product_code": ["PRD-1001", "PRD-1002", "PRD-1003", "PRD-1004 ", "PRD-1005"],
    "category": ["Electronics", "Electronics", "accessories", "Accessories", "Electronics"],
    "unit_price": ["799", "1,499", "199", "89900", "2499"], # 89900 is a price outlier
    (data-entry error)
    "qty": [118, 40, 305, 58, 75],
})

online_df = pd.DataFrame({
    "name": ["Wireless Mouse", "Bluetooth Speaker", "USB-C Cable", "Laptop Stand", "Gaming
Keyboard"],
    "code": ["PRD1001", "PRD-1002", "PRD_1003", "PRD-1004", "PRD-1005"],
    "category": ["Electronics", "Electronics", "Accessories", "Accessories", "electronics"],
    "price": [799, 1499, 199, 899, 2499],
    "stock_qty": [122, 42, 298, 61, 70],
})

# -----
# 2. Helper functions: RegEx validation, fuzzy matching, standardization
# -----
CODE_PATTERN = re.compile(r"^(PRD-\d{4})$")

def clean_product_code(code: str) -> str:
    """Standardize a product code to the canonical PRD-#### form."""
    if pd.isna(code):
        return np.nan
    digits = re.sub(r"[^0-9]", "", str(code))
    return f"PRD-{digits}" if digits else np.nan

def is_valid_code(code: str) -> bool:
    return bool(CODE_PATTERN.match(str(code)))
```

```

def clean_price(value) -> float:
    """Strip currency symbols/commas and convert to float."""
    cleaned = re.sub(r"^[^d.]", "", str(value))
    return float(cleaned) if cleaned else np.nan

def similar(a: str, b: str) -> float:
    """Fuzzy-match ratio between two strings (0-1)."""
    return SequenceMatcher(None, a.lower().strip(), b.lower().strip()).ratio()

def fuzzy_group_names(names, threshold=0.85):
    """Group near-duplicate product names, keeping the first spelling seen."""
    canonical = []
    mapping = {}
    for n in names:
        n_norm = re.sub(r"\s+", " ", str(n)).strip()
        match = next((c for c in canonical if similar(c, n_norm) >= threshold), None)
        if match:
            mapping[n] = match
        else:
            canonical.append(n_norm)
            mapping[n] = n_norm
    return mapping

# -----
# 3. Standardize each source dataset to a common schema
# -----
supplier_std = supplier_df.rename(columns={"prod_name": "name", "prod_code": "code",
                                           "price": "price", "stock": "stock"})
warehouse_std = warehouse_df.rename(columns={"product_name": "name", "product_code": "code",
                                              "unit_price": "price", "qty": "stock"})
online_std = online_df.rename(columns={"stock_qty": "stock"})

combined = pd.concat([supplier_std, warehouse_std, online_std], ignore_index=True)

# Clean codes and prices with RegEx-based helpers
combined["code"] = combined["code"].apply(clean_product_code)
combined["code_valid"] = combined["code"].apply(is_valid_code)
combined["price"] = combined["price"].apply(clean_price)
combined["category"] = combined["category"].str.strip().str.title()

# Fuzzy-match product names to a single canonical spelling
name_map = fuzzy_group_names(combined["name"])
combined["name"] = combined["name"].map(name_map)
combined["name"] = combined["name"].str.replace(r"\s+", " ", regex=True).str.strip()

# -----
# 4. Outlier detection on price (IQR method) per product
# -----
def price_outlier_mask(prices: pd.Series) -> pd.Series:
    """Flag a price as an outlier if it deviates from the group median
    by more than 3x (robust for the small per-product sample sizes here)."""
    med = prices.median()
    return (prices > med * 3) | (prices < med / 3)

combined["price_outlier"] = combined.groupby("name")["price"].transform(price_outlier_mask)

# Drop rows whose price was flagged as an outlier (bad data entry, e.g. 89900)
clean_rows = combined[~combined["price_outlier"]].copy()

# -----
# 5. Duplicate detection & removal (same name + code -> keep first, average stock)
# -----
master = (
    clean_rows.groupby(["name", "code", "category"], as_index=False)
    .agg(price=("price", "median"), stock=("stock", "mean"))
)
master["stock"] = master["stock"].round().astype(int)
master = master.sort_values("code").reset_index(drop=True)

```

```

# -----
# 6. Save the final product master file (CSV + JSON)
# -----
master.to_csv("product_master.csv", index=False)
master.to_json("product_master.json", orient="records", indent=2)

# -----
# 7. Reusable cleanup function to test on a NEW dataset
# -----
def cleanup_pipeline(df: pd.DataFrame, name_col: str, code_col: str,
                    category_col: str, price_col: str, stock_col: str) -> pd.DataFrame:
    """Reusable end-to-end cleaner: standardize schema, clean fields,
    fuzzy-match names, drop price outliers, and remove duplicates."""
    d = df.rename(columns={name_col: "name", code_col: "code",
                          category_col: "category", price_col: "price", stock_col:
"stock"}).copy()
    d["code"] = d["code"].apply(clean_product_code)
    d["price"] = d["price"].apply(clean_price)
    d["category"] = d["category"].str.strip().str.title()
    nmap = fuzzy_group_names(d["name"])
    d["name"] = d["name"].map(nmap).str.replace(r"\s+", " ", regex=True).str.strip()
    d = d.groupby(["name", "code", "category"], as_index=False).agg(
        price=("price", "median"), stock=("stock", "mean"))
    d["stock"] = d["stock"].round().astype(int)
    return d

new_data = pd.DataFrame({
    "item": ["Desk Lamp", "Desk Lamp", "Webcam HD"],
    "sku": ["PRD-2001", "prd2001", "PRD-2002"],
    "cat": ["Electronics", "electronics", "Electronics"],
    "cost": [599, 599, 1299],
    "units": [30, 32, 15],
})
new_master = cleanup_pipeline(new_data, "item", "sku", "cat", "cost", "units")

# -----
# 8. Print summary output
# -----
print("=== Combined raw records (before cleaning) ===")
print(len(combined), "rows from 3 sources\n")

print("=== Records dropped as price outliers ===")
print(combined[combined["price_outlier"]][["name", "code", "price"]].to_string(index=False),
      "\n")

print("=== FINAL PRODUCT MASTER DATASET ===")
print(master.to_string(index=False), "\n")

print("=== Saved files ===")
print("product_master.csv, product_master.json\n")

print("=== Reusable pipeline tested on a NEW dataset ===")
print(new_master.to_string(index=False))

```

Output

```

=== Combined raw records (before cleaning) ===
15 rows from 3 sources

=== Records dropped as price outliers ===
   name   code  price
Laptop Stand PRD-1004 89900.0

=== FINAL PRODUCT MASTER DATASET ===
   name   code  category  price  stock
Wireless Mouse PRD-1001 Electronics  799.0   120
Bluetooth Speaker PRD-1002 Electronics 1499.0   42
USB-C Cable PRD-1003 Accessories  199.0  301
Laptop Stand PRD-1004 Accessories  899.0   60
Gaming Keyboard PRD-1005 Electronics 2499.0   72

```

```

=== Saved files ===
product_master.csv, product_master.json

=== Reusable pipeline tested on a NEW dataset ===
   name    code  category  price  stock
Desk Lamp PRD-2001 Electronics  599.0    31
Webcam HD PRD-2002 Electronics 1299.0    15

```

Question 5 – Healthcare Patient Dataset Integration

A hospital stores patient information in **Registration, Laboratory, and Billing datasets**. Patient names may contain spelling errors, phone numbers use different formats, emails may be invalid, dates have different formats, and duplicate patient records exist across departments.

Task: Develop a Python program to clean and integrate the three datasets. Identify bad data and missing values, validate fields using **RegEx**, **detect duplicates**, **perform fuzzy matching**, **standardize dates and categorical values**, **identify numerical outliers**, and **merge the datasets**. Save the final cleaned dataset and create a script that can be tested on newly received patient data.

Answer:

Python Code

```

import re
import numpy as np
import pandas as pd
from difflib import SequenceMatcher

# -----
# 1. Sample source datasets (simulating Registration / Laboratory / Billing)
# -----
registration_df = pd.DataFrame({
    "patient_name": ["Ananya Rao", "Ananyaa Rao", "Vikram Singh", "Meera Iyer", "Suresh Kumar"],
    "phone": ["9876543210", "98765-43210", "9123456780", "9012345678", "9988776655"],
    "email": ["ananya.rao@mail.com", "ananya.rao@mail.com", "vikramsingh@mail.com", "meera.iyer@mail.com", "suresh_k@mail"],
    "dob": ["12-05-1990", "12/05/1990", "1985-11-02", "23-01-1978", "07/09/1995"],
    "gender": ["F", "F", "M", "F", "M"],
})

laboratory_df = pd.DataFrame({
    "name": ["Ananya Rao", "Vikram Singh", "Meera Iyer", "Suresh Kumar", "Divya Nair"],
    "contact": ["9876543210", "9123456780", "9012345678", "99887 76655", "9000011111"],
    "test_date": ["2024/05/12", "2024-05-14", "14-05-2024", "2024-05-15", "2024-05-16"],
    "glucose_level": [95, 110, 96, 620, 102], # 620 mg/dL (Suresh) is a clear outlier (data-entry error)
})

billing_df = pd.DataFrame({
    "patient": ["Ananya Rao", "Vikram Singh", "Meera Iyer", "Suresh Kumar", "Divya Nair"],
    "phone_no": ["9876543210", "9123456780", "9012345678", "9988776655", "9000011111"],
    "bill_amount": ["2,500", "4800", "1500", "3200.5", "99999"], # 99999 is a billing outlier
    "billing_date": ["12-05-2024", "14/05/2024", "2024-05-14", "15-05-2024", "16-05-2024"],
})

# -----
# 2. Helper functions: RegEx validation, fuzzy matching, standardization
# -----
PHONE_PATTERN = re.compile(r"^[6-9]\d{9}$")
EMAIL_PATTERN = re.compile(r"^[w.-]+@[w.-]+\.[a-zA-Z]{2,}$")

def clean_phone(value) -> str:

```

```

    digits = re.sub(r"\D", "", str(value))
    return digits[-10:] if len(digits) >= 10 else np.nan

def is_valid_phone(value) -> bool:
    return bool(PHONE_PATTERN.match(str(value)))

def is_valid_email(value) -> bool:
    return bool(EMAIL_PATTERN.match(str(value)))

def parse_date(value):
    """Standardize mixed date formats (DD-MM-YYYY, DD/MM/YYYY, YYYY-MM-DD, YYYY/MM/DD) to
    YYYY-MM-DD."""
    for fmt in ("%d-%m-%Y", "%d/%m/%Y", "%Y-%m-%d", "%Y/%m/%d"):
        try:
            return pd.to_datetime(value, format=fmt).strftime("%Y-%m-%d")
        except (ValueError, TypeError):
            continue
    return np.nan

def similar(a: str, b: str) -> float:
    return SequenceMatcher(None, a.lower().strip(), b.lower().strip()).ratio()

def fuzzy_group_names(names, threshold=0.85):
    canonical, mapping = [], {}
    for n in names:
        n_norm = re.sub(r"\s+", " ", str(n)).strip()
        match = next((c for c in canonical if similar(c, n_norm) >= threshold), None)
        if match:
            mapping[n] = match
        else:
            canonical.append(n_norm)
            mapping[n] = n_norm
    return mapping

# -----
# 3. Standardize each source to a common schema and merge on cleaned phone
# -----
reg = registration_df.rename(columns={"patient_name": "name"}).copy()
reg["phone"] = reg["phone"].apply(clean_phone)
reg["dob"] = reg["dob"].apply(parse_date)
reg["email_valid"] = reg["email"].apply(is_valid_email)

lab = laboratory_df.rename(columns={"contact": "phone"}).copy()
lab["phone"] = lab["phone"].apply(clean_phone)
lab["test_date"] = lab["test_date"].apply(parse_date)

bill = billing_df.rename(columns={"patient": "name", "phone_no": "phone"}).copy()
bill["phone"] = bill["phone"].apply(clean_phone)
bill["billing_date"] = bill["billing_date"].apply(parse_date)
bill["bill_amount"] = bill["bill_amount"].astype(str).str.replace(",", "",
regex=False).astype(float)

# Fuzzy-match every name across the 3 sources to one canonical spelling
all_names = pd.concat([reg["name"], lab["name"], bill["name"]])
name_map = fuzzy_group_names(all_names)
for d in (reg, lab, bill):
    d["name"] = d["name"].map(name_map)

# Merge on the cleaned 10-digit phone number (a reliable join key across systems)
merged = reg.merge(lab[["phone", "test_date", "glucose_level"]], on="phone", how="outer")
merged = merged.merge(bill[["phone", "bill_amount", "billing_date"]], on="phone", how="outer")
merged["name"] = merged["name"].fillna(merged["phone"].map(dict(zip(bill["phone"],
bill["name"]))))

# -----
# 4. Missing value handling
# -----
merged["gender"] = merged["gender"].fillna("Unknown")

```

```

merged["glucose_level"] = merged["glucose_level"].fillna(merged["glucose_level"].median())
merged["bill_amount"] = merged["bill_amount"].fillna(merged["bill_amount"].median())

# -----
# 5. Numerical outlier detection (glucose level & bill amount) via Z-score
# -----
def zscore_outlier(series: pd.Series, threshold: float = 2.0) -> pd.Series:
    z = (series - series.mean()) / series.std(ddof=0)
    return z.abs() > threshold

merged["glucose_outlier"] = zscore_outlier(merged["glucose_level"])
merged["bill_outlier"] = zscore_outlier(merged["bill_amount"])

# Cap outliers to the nearer non-outlier bound instead of dropping the patient record
glucose_ok = merged.loc[~merged["glucose_outlier"], "glucose_level"]
merged.loc[merged["glucose_outlier"], "glucose_level"] = glucose_ok.median()
bill_ok = merged.loc[~merged["bill_outlier"], "bill_amount"]
merged.loc[merged["bill_outlier"], "bill_amount"] = bill_ok.median()

# -----
# 6. Duplicate detection & removal (same phone -> one patient record)
# -----
final = merged.drop_duplicates(subset=["phone"]).reset_index(drop=True)
final = final[["name", "phone", "email", "dob", "gender", "test_date",
               "glucose_level", "billing_date", "bill_amount"]]

# -----
# 7. Save the final cleaned dataset
# -----
final.to_csv("patient_master.csv", index=False)
final.to_json("patient_master.json", orient="records", indent=2)

# -----
# 8. Reusable cleanup function, tested on newly received patient data
# -----
def cleanup_patient_batch(df: pd.DataFrame, name_col: str, phone_col: str, date_col: str) ->
pd.DataFrame:
    """Reusable cleaner for a single newly received patient batch:
    fixes phone format, validates it, standardizes the date, and fuzzy-groups names."""
    d = df.rename(columns={name_col: "name", phone_col: "phone", date_col: "date"}).copy()
    d["phone"] = d["phone"].apply(clean_phone)
    d["phone_valid"] = d["phone"].apply(is_valid_phone)
    d["date"] = d["date"].apply(parse_date)
    nmap = fuzzy_group_names(d["name"])
    d["name"] = d["name"].map(nmap)
    return d.drop_duplicates(subset=["phone"]).reset_index(drop=True)

new_patients = pd.DataFrame({
    "pname": ["Rahul Verma", "Rahul Verma", "Sneha Pillai"],
    "mobile": ["98765-11111", "9876511111", "91234 22222"],
    "visit_date": ["01-06-2024", "2024-06-01", "03/06/2024"],
})
new_clean = cleanup_patient_batch(new_patients, "pname", "mobile", "visit_date")

# -----
# 9. Print summary output
# -----
print("=== Merged records before outlier correction ===")
print(merged[["name", "phone", "glucose_level", "bill_amount",
               "glucose_outlier", "bill_outlier"]].to_string(index=False), "\n")

print("=== FINAL CLEANED & INTEGRATED PATIENT DATASET ===")
print(final.to_string(index=False), "\n")

print("=== Saved files ===")
print("patient_master.csv, patient_master.json\n")

print("=== Reusable pipeline tested on newly received patient data ===")
print(new_clean.to_string(index=False))

```

Output

```
=== Merged records before outlier correction ===
      name      phone  glucose_level  bill_amount  glucose_outlier  bill_outlier
Divya Nair 9000011111          102      2500.0          False          True
Meera Iyer 9012345678           96      1500.0          False          False
Vikram Singh 9123456780        110      4800.0          False          False
Ananya Rao 9876543210           95      2500.0          False          False
Ananya Rao 9876543210           95      2500.0          False          False
Suresh Kumar 9988776655         96      3200.5          True          False

=== FINAL CLEANED & INTEGRATED PATIENT DATASET ===
      name      phone      email      dob  gender  test_date  glucose_level
billing_date  bill_amount
Divya Nair 9000011111          NaN      NaN Unknown 2024-05-16          102
2024-05-16      2500.0
Meera Iyer 9012345678  meera.iyer@mail.com 1978-01-23      F 2024-05-14          96
2024-05-14      1500.0
Vikram Singh 9123456780 vikramsingh@mail.com 1985-11-02      M 2024-05-14          110
2024-05-14      4800.0
Ananya Rao 9876543210  ananya.rao@mail.com 1990-05-12      F 2024-05-12          95
2024-05-12      2500.0
Suresh Kumar 9988776655  suresh_k@mail 1995-09-07      M 2024-05-15          96
2024-05-15      3200.5

=== Saved files ===
patient_master.csv, patient_master.json

=== Reusable pipeline tested on newly received patient data ===
      name      phone      date  phone_valid
Rahul Verma 9876511111 2024-06-01      True
Sneha Pillai 9123422222 2024-06-03      True
```